

ETLens AI

ETL Script Documentation Generator

1. Project Overview

ETLens is an intelligent documentation generator that automatically reads Python and SQL ETL (Extract, Transform, Load) scripts and produces comprehensive documentation. The system eliminates the manual effort of documenting complex data pipelines by leveraging AI to analyze code structure, explain business logic, generate visual flow diagrams, and export professional PDF reports.

Problem Statement

Data engineering teams spend significant time manually documenting ETL scripts. These scripts often lack proper documentation, making it difficult for new team members to understand data flows, business logic, and dependencies. ETLens solves this by automating the entire documentation process.

Solution

ETLens automatically:

- Parses Python and SQL ETL scripts using AST and sqlparse
- Generates detailed technical documentation via a local LLM (Llama3)
- Explains the business purpose of each script in plain language
- Creates visual flow diagrams using Graphviz
- Performs dependency impact analysis using NetworkX
- Answers questions about scripts using RAG (FAISS + SentenceTransformers)
- Exports complete documentation as a professional PDF report

2. Technology Stack

Component	Technology	Purpose
-----------	------------	---------

Backend API	FastAPI (Python)	REST API server for all processing endpoints
Frontend UI	Streamlit	Interactive 6-page web interface
Local LLM	Ollama + Llama3	AI documentation and business explanation generation
Python Parser	AST (Abstract Syntax Tree)	Parses Python ETL scripts structurally
SQL Parser	sqlparse library	Parses and analyzes SQL ETL scripts
RAG Pipeline	FAISS + SentenceTransformers	Vector search for Q&A on scripts
Flow Diagrams	Graphviz + NetworkX	Visual dependency and flow diagrams
PDF Export	fpdf2	Professional PDF report generation
Database	SQLite	Stores parsed results and documentation
AI Framework	LangChain	LLM orchestration and prompt management

3. System Architecture

Architecture Overview

ETLens follows a clean 3-tier architecture:

- Presentation Layer: Streamlit frontend with 6 dedicated pages
- Business Logic Layer: FastAPI backend with modular processing components
- Data Layer: SQLite database with file-based cache for pre-generated outputs

Processing Pipeline

Step	Module	Description
------	--------	-------------

1	parser/python_parser.py	AST-based parsing of Python ETL scripts
2	parser/sql_parser.py	sqlparse-based parsing of SQL scripts
3	ai/doc_generator.py	Generates technical documentation via LLM
4	ai/business_explainer.py	Explains business purpose in plain language
5	diagram/flow_diagram.py	Creates Graphviz PNG flow diagrams
6	impact/impact_analysis.py	NetworkX dependency graph analysis
7	ai/rag_pipeline.py	FAISS vector store for script Q&A
8	export/pdf_exporter.py	Generates PDF documentation report

4. Key Features

Feature	Description
Automatic Parsing	Supports both Python (.py) and SQL (.sql) ETL scripts with deep structural analysis
AI Documentation	Generates comprehensive technical docs including functions, tables, transformations
Business Explanation	Translates technical ETL logic into business-friendly language
Flow Diagrams	Visual Graphviz diagrams showing data flow and transformations
Impact Analysis	NetworkX dependency graph showing which tables/functions depend on each other
RAG-based Q&A	Ask natural language questions about any uploaded script
PDF Export	One-click professional PDF report with all documentation
Cache System	Pre-generated outputs for instant demo without LLM delays

Template Fallback	Instant template-based generation for new scripts when LLM is slow
-------------------	--

5. Project Structure

File / Folder	Description
backend/main.py	FastAPI application with all API endpoints
frontend/app.py	Streamlit UI with 6 pages
parser/python_parser.py	AST-based Python script parser
parser/sql_parser.py	sqlparse-based SQL script parser
ai/llm_client.py	Ollama HTTP client (timeout=600s)
ai/doc_generator.py	LLM-based documentation generator
ai/business_explainer.py	LLM-based business purpose explainer
ai/rag_pipeline.py	FAISS + SentenceTransformers RAG pipeline
diagram/flow_diagram.py	Graphviz flow diagram generator
impact/impact_analysis.py	NetworkX impact analysis module
export/pdf_exporter.py	fpdf2 PDF report exporter
run_pregenerate.py	Pre-generates all outputs for instant demo
output/pregenerated_cache.json	Cache of pre-generated documentation
Document/	Project documents and sample ETL scripts

6. How to Run

Prerequisites

- Python 3.10+
- Node.js (optional, for frontend dev)
- Ollama installed with Llama3 model pulled
- Graphviz installed on system

Setup Steps

- Clone the repository and navigate to project folder
- Activate virtual environment: `.\venv\Scripts\activate`
- Install dependencies: `pip install -r requirements.txt`
- Start Ollama: `ollama serve` (in separate terminal)
- Pre-generate cache: `python run_pregenerate.py`
- Start backend: `uvicorn backend.main:app --reload --port 8000`
- Start frontend: `streamlit run frontend/app.py`
- Open browser: `http://localhost:8501`

7. Challenges & Solutions

Challenge	Solution
LLM slow on local CPU	Implemented pre-generation cache + instant template fallback
Large file push to GitHub	Used Git LFS for video; optimized other assets
Complex SQL parsing	Used sqlparse library with custom logic for ETL-specific patterns
RAG accuracy	Tuned SentenceTransformers model and FAISS index parameters
PDF formatting	Used fpdf2 with custom templates for professional output

8. Conclusion

ETLens successfully automates ETL script documentation, reducing documentation time from hours to seconds. The system combines modern AI capabilities (local LLM, RAG, vector search) with robust software engineering (FastAPI, Streamlit, caching) to deliver a production-ready documentation tool. The hackathon prototype demonstrates clear business value for any data engineering team managing complex ETL pipelines.